

Bonn Mobility Analytics Platform

Projektbeschreibung und grober Leitfaden für die Umsetzung mit Cursor

Ziel: Eine öffentlich erreichbare, cloud-native Mobility-Plattform für den Raum Bonn/Rhein-Sieg, die reale Echtzeitdaten des Welo-Fahrradmietsystems ingestiert, aufbereitet, analysiert und über eine moderne Website visualisiert. Das Projekt soll gleichzeitig als ernsthaftes Portfolio- und DevOps-Projekt dienen.

1. Projektidee

Die Plattform soll nicht einfach eine weitere Karte mit verfügbaren Fahrrädern sein. Der Mehrwert liegt in einer zusätzlichen Analyseebene über den Live-Daten: aktuelle Verfügbarkeit, historische Entwicklungen, Stationsauslastung, problematische Stationen, Trends und später optional Prognosen.

Die Anwendung soll einen realistischen Produktionsbetrieb simulieren: externe Echtzeitdaten werden regelmäßig abgeholt, verarbeitet und gespeichert. Nutzer greifen auf die eigene API zu. Unterschiedliche Backend-Workloads werden als voneinander getrennte Services betrieben und können unabhängig skaliert werden.

2. Kernfunktion der Plattform

- Live Mobility: aktuelle Fahrzeuge, Stationen und Verfügbarkeiten im Raum Bonn/Rhein-Sieg.
- Historie: eigene Zeitreihen aus regelmäßig gespeicherten Daten-Snapshots.
- Analytics: Auslastung, Verfügbarkeit, Trends und Vergleich von Stationen.
- Mobility Health: Bewertung von Stationen anhand von Verfügbarkeit, Schwankungen und historischen Mustern.
- Optional später: einfache Prognosen, z. B. ob eine Station voraussichtlich bald leer oder voll wird.
- Öffentliche Website: interaktive Karte, Statistiken und verständliche Visualisierungen.

3. Grobe Zielarchitektur

Externe Datenquelle
Welo / GBFS Echtzeitdaten

↓

Kubernetes Cluster

- Welo Ingestion Service
- Mobility API Service
- Analytics Service / Worker
- optional Notification Service
- optional API Gateway / Ingress

↓

Managed Data Layer
Supabase / PostgreSQL

↓

Die genaue technische Architektur wird später gemeinsam ausgearbeitet. Die oben genannten Komponenten sind zunächst Leitplanken und keine endgültig festgelegten Implementierungsdetails.

4. Services - grobe Verantwortung

Welo Ingestion Service

Ruft den öffentlichen Welo-Feed regelmäßig ab, validiert die Antwort, normalisiert relevante Daten und schreibt aktuelle bzw. historische Daten in die Datenhaltung. Der Service soll robust gegen temporäre Fehler und externe API-Ausfälle sein.

Mobility API Service

Stellt eine eigene API für das Frontend bereit. Das Frontend soll nicht direkt auf die externe Welo-API zugreifen. Die API liefert aktuelle Fahrzeuge, Stationen, Verfügbarkeiten und später Analytics.

Analytics Service / Worker

Berechnet aus historischen Daten Kennzahlen und Trends. Der Service soll unabhängig von der API skalierbar sein, weil Analytics-Workloads andere Ressourcenanforderungen haben können.

Optional Notification Service

Kann später Alerts oder Benachrichtigungen verarbeiten, beispielsweise wenn eine Station einen kritischen Verfügbarkeitswert erreicht.

5. Warum Kubernetes?

Kubernetes soll nicht nur eingebaut werden, um ein Buzzword abzuhaken. Das Projekt soll zeigen, warum Container-Orchestrierung bei mehreren unabhängig betriebenen Workloads sinnvoll sein kann.

- Unabhängiges Deployment der Services.
- Self-healing: ausgefallene Pods werden ersetzt.
- Horizontale Skalierung des API-Services bei hoher Nutzerlast.
- Unterschiedliche Skalierung von API, Ingestion und Analytics.
- Rolling Updates ohne vollständigen Ausfall.
- Service Discovery und interne Kommunikation.
- Ressourcenlimits und Requests.
- Health Checks / Readiness / Liveness.
- Später optional Horizontal Pod Autoscaling.

Ein wichtiger Teil des Projekts soll daher ein realistischer Lasttest sein: beispielsweise simulierte Nutzer, die gleichzeitig die Karte, Stationsdaten und Analytics-Endpunkte abfragen. Dabei soll beobachtet werden, wie sich CPU, Memory, Request-Latenz und Pod-Anzahl verändern.

6. DevOps-Stack - geplante Bereiche

- Docker: jeder Backend-Service als reproduzierbares Container-Image.

- Kubernetes: Deployment, Services, Ingress, ConfigMaps/Secrets, Health Checks und Skalierung.
- Terraform: Infrastructure as Code für die benötigte Infrastruktur.
- Helm: optional für reproduzierbare Kubernetes-Deployments.
- CI/CD: automatisierte Tests, Docker Build, Image Push und Deployment.
- Prometheus: Metriken und Monitoring.
- Grafana: Dashboards für System- und Applikationsmetriken.
- Redis: optional als Cache für häufig angefragte Daten.
- Message Queue: optional, z. B. RabbitMQ oder Kafka, falls Event Processing sinnvoll wird.
- Load Testing: z. B. k6, um realistische Nutzerlast zu simulieren.
- Logging/Tracing: später ergänzen, wenn die Basis stabil läuft.

7. Datenfluss

```
Welo
↓
Ingestion Service
↓
Validierung / Normalisierung
↓
Persistierung bzw. Event-Verarbeitung
↓
PostgreSQL / Supabase
↓
Mobility API
↓
Vercel Frontend
↓
Nutzer
```

Zusätzlich können historische Snapshots gespeichert werden. Dadurch entsteht mit der Zeit ein eigener Datensatz, aus dem Analytics und Visualisierungen erzeugt werden können.

8. Frontend

- Interaktive Karte des Bonner/Rhein-Sieg-Raums.
- Live-Anzeige von Fahrzeugen und Stationen.
- Station-Detailseite mit aktueller Verfügbarkeit.
- Historische Verfügbarkeitskurven.
- Analytics-Dashboard.
- Mobility-Health-Indikatoren.
- Optional Nutzerkonto/Favoriten, falls dies später technisch sinnvoll ist.

Das bestehende Frontend kann weiterhin auf Vercel betrieben werden. Das Kubernetes-Cluster muss nicht den Frontend-Teil hosten.

9. Bestehende Infrastruktur

Ausgangspunkt ist eine bereits vorhandene Architektur mit Vercel für das Frontend und Supabase/PostgreSQL für die Datenbank. Diese Komponenten sollen zunächst weiterverwendet werden. Kubernetes übernimmt primär die Backend-Services und Worker.

10. Entwicklung in Phasen

Phase 1 - funktionierender Prototyp

- Welo-Daten abrufen.
- Datenmodell definieren.
- PostgreSQL/Supabase anbinden.
- Einfache API bauen.
- Live-Karte im Frontend.

Phase 2 - Service-Struktur

- Ingestion und API trennen.
- Analytics-Service ergänzen.
- Services containerisieren.
- Lokale Entwicklung mit Docker Compose.

Phase 3 - Kubernetes

- Cluster aufsetzen.
- Deployments und Services erstellen.
- Ingress konfigurieren.
- Secrets sauber verwalten.
- Health Checks und Resource Limits.
- Rolling Updates.

Phase 4 - CI/CD & Infrastructure as Code

- GitHub Actions.
- Automatisierte Tests.
- Docker Image Build.
- Container Registry.
- Terraform.
- Automatisiertes Deployment.

Phase 5 - Observability & Skalierung

- Prometheus.
- Grafana.
- Application Metrics.
- Load Testing mit k6.

- Autoscaling.
- Fehler-/Ausfallszenarien testen.

11. Was das Projekt demonstrieren soll

Das Projekt soll einem technischen Interviewer zeigen, dass der Entwickler nicht lediglich einzelne Technologien aus Tutorials kennt, sondern deren Einsatz in einem zusammenhängenden System versteht.

- Warum Services getrennt werden.
- Warum Container verwendet werden.
- Wann Kubernetes einen Mehrwert bietet.
- Wie sich unterschiedliche Workloads skalieren lassen.
- Wie externe Echtzeitdaten robust verarbeitet werden.
- Wie CI/CD und Infrastructure as Code eingesetzt werden.
- Wie ein System überwacht und unter Last getestet wird.
- Wie Fehler und Ausfälle behandelt werden.

12. Leitlinie für Cursor

Cursor soll das Projekt inkrementell entwickeln und nicht sofort eine riesige Architektur generieren. Vor jeder größeren Implementierung sollen Architektur, Datenmodell und Schnittstellen kurz festgelegt werden. Die einzelnen Services sollen klar abgegrenzte Verantwortlichkeiten besitzen. Keine Technologie soll nur zum Selbstzweck eingebaut werden.

Technische Details wie Programmiersprache, Frameworks, Datenbankschema, konkreter Kubernetes-Provider, Message Broker, API-Spezifikation, Authentifizierung, Domain-Modell und Deployment-Strategie werden in späteren Schritten gemeinsam festgelegt.

Oberstes Ziel: Ein tatsächlich funktionierendes, öffentlich demonstrierbares System mit realen Daten, reproduzierbarer Infrastruktur und nachvollziehbarer DevOps-Architektur – nicht bloß eine Sammlung von Demo-Services.